Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

One job per function

Today
Welcome to your next lesson: One job per function.

I'm preparing your personalized lesson based on what you've already learned, what's next, and how you learn best.

Step by step, you're moving forward.

Imagine a giant Python function called process_coffee_order() ☕.

Inside it, all mixed together, you see:

Asking the user what they want
Figuring out the price
Applying a discount
Printing a receipt
Updating a daily total

All in one long function. Any small change (like changing the receipt text) risks breaking something else. That's the pain we want to avoid.

In this lesson, we'll learn a simple rule: one function should do one clear job.

To warm up: when you picture that mega process_coffee_order() doing all those things, does it feel easier to:

change safely, or
accidentally break things?

Just answer in your own words, then we'll build from there.

accidentaly break things

Exactly 💯 — that big mixed-up function is a recipe for accidental breakage.

Let's zoom in on the core idea, one step at a time.

Single-responsibility principle (for functions):

A function should have one clear job. If it's doing several different kinds of things, it's breaking the rule.

In the coffee example, these are actually different jobs:

Talking to the user (input)
Doing math (price, discount)
Showing results (printing receipt)
Tracking totals (updating some stored value)

Putting them all in one function is like having one employee who is barista + cashier + manager + cleaner at the same time. Things get messy.

Quick check-in:

If you see a Python function that both calculates a value and prints a full user-facing message, does that sound like:

one job, or
more than one job?

Answer in your own words, then we'll look at a tiny code example next.

Sounds like more than one job

Right on — that is more than one job 🎯

Let me show you a tiny Python example to make this concrete.

Python

```
def final_price_message(price, discount_percent):
    discounted = price - (price * discount_percent / 100)
    print("Your final price is", discounted)
```

In this one function we have:

Math work: discounted = ...
User message: print("Your final price is", discounted)

So this function is:

calculating, and
talking to the user.

That's two jobs in one place → not ideal.

Make sense so far: a function that both computes something and prints to the user is doing multiple jobs?

Just answer yes / no (or "kind of") and then I'll walk you through how we'd split this into two smaller, clearer functions.

Yes

Great 👍

Now let's see how to fix that two-jobs-in-one function by splitting it.

We take this:

Python

```
def final_price_message(price, discount_percent):
```

```
    discounted = price - (price * discount_percent / 100)
    print("Your final price is", discounted)
```

and turn it into two functions with one job each:

One function that only calculates and returns the number.
Another function that only prints the message.

Visually:

calculate_discounted_price → 🎯 job: math only → returns a number
show_final_price → 🎯 job: message only → prints using that number
Before I show the code split, quick check:

Do you want to see a little visual diagram of "before vs after" for this function split, to make it clearer?

Reply with yes or no.

No, let's continue

Nice, let's keep going step by step 👍

Here's how we split that function into one job per function:

Python

```
def calculate_discounted_price(price, discount_percent):
    discounted = price - (price * discount_percent / 100)
    return discounted   # ☑ only calculates and returns


def show_final_price(discounted_price):
    print("Your final price is", discounted_price)  # ☑ only prints
```

Now:

calculate_discounted_price → math only, no printing
show_final_price → printing only, no math

A third function (or just some top-level code) could coordinate:

Python

```
discounted = calculate_discounted_price(100, 10)
show_final_price(discounted)
```

Does this step-by-step split make sense: one helper for the calculation, another for the message?

If anything feels fuzzy, tell me which part: the return, the split, or how they work together.

it makes sense

Good 👍 Since the idea is clear, let's practice spotting when a function is doing too much.

Look at this imaginary function:

Python

```
def process_order(name, quantity, price_each):
    total = quantity * price_each
    print("Customer:", name)
    print("Total items:", quantity)
```

```
    print("Total cost:", total)
    return total
```

Inside it we have:

math: total = quantity * price_each
prints about the customer
prints about the total
and it also returns total

So here’s the key idea:

If a function is calculating something and also handling how it’s shown to the user, that usually means more than one responsibility.

I want to check your feel for this:

In process_order above, what do you think is the main job it should focus on if we followed “one job per function”?
You can answer in your own words, something like:

“just do the math”
or “just print stuff”
or something else you think fits.
its name and the parameters its passed sugest its main task is to calculate a total

Nice reasoning, Ron 👌 That’s exactly how I want you to think: the name and parameters hint that its main job is to compute the total.

So if we apply “one job per function” to:

Python

```
def process_order(name, quantity, price_each):
    total = quantity * price_each
    print("Customer:", name)
    print("Total items:", quantity)
    print("Total cost:", total)
    return total
```

we’d say:

It’s doing too much (math + printing report)
A cleaner design would be:
one function focused on computing the total
another focused on printing / showing the order details
Next, I’d like to walk you through a refactor like that, then have you try a similar split yourself.

Are you up for doing a small refactor example next?

Yes, let's try it

Love it. Let’s refactor that process_order together, step by step 🛠

We start with the original:

Python

```
def process_order(name, quantity, price_each):
    total = quantity * price_each
    print("Customer:", name)
```

```
    print("Total items:", quantity)
    print("Total cost:", total)
    return total
```

Goal: one job per function.

A clean split would look like this (conceptually):

A function that only calculates and returns the total:

Python

```
def calculate_total(quantity, price_each):
    total = quantity * price_each
    return total
```

A function that only prints the order details:

Python

```
def show_order_summary(name, quantity, total):
    print("Customer:", name)
    print("Total items:", quantity)
    print("Total cost:", total)
```

Then some coordinating code:

Python

```
total = calculate_total(3, 5.0)
show_order_summary("Ron", 3, total)
```

Now:

calculate_total = one clear responsibility: math

show_order_summary = one clear responsibility: display

Quick check:

Does this split into calculate_total and show_order_summary feel clear to you, or is there any part of this refactor you want me to walk through more slowly before we move to your own practice example?

It's clear to me

Nice. Then you're ready to try your own split 🖋

I'll give you a function that's doing more than one job, and your task will be to split it into two helpers.

Here's the idea we'll use:

We start with a single function like:

Python

```
def grade_calculator(scores):
    # 1) calculate numeric average
    # 2) decide letter grade (A, B, C, etc.)
    # 3) maybe print something
```

Your goal will be to turn this into:

numeric_average(scores) → only compute and return the average number

letter_grade(average) → only decide and return the letter (like "A", "B")

You'll write this in the code editor, not here in chat.

Step 1:

Open the code editor (</>), and I'll send you the exact grade_calculator function to

refactor next.

Reply “ready in editor” when you have it open.

ready in editor

Awesome, Ron 🎯

Here’s the starting function for you to refactor in the editor:

Python

```
def grade_calculator(scores):
    total = 0
    for score in scores:
        total += score
    average = total / len(scores)

    if average >= 90:
        letter = "A"
    elif average >= 80:
        letter = "B"
    elif average >= 70:
        letter = "C"
    elif average >= 60:
        letter = "D"
    else:
        letter = "F"

    print("Average:", average, "Letter grade:", letter)
    return average, letter
```

Your task in the editor:

Create numeric_average(scores)

does only the average math
returns the average (number)
Create letter_grade(average)

does only the if/elif logic
returns the letter ("A", "B", etc.)
Then make grade_calculator(scores):

call numeric_average(scores)
call letter_grade(average)
print the final line (if you want)
return both values

Try it now in the editor, run it to be sure it works, then hit Submit code so I can review it.

Python

```
score_list = [85, 95, 100, 75]
def numeric_average(scores):
    total = 0
    for score in scores:
        total += score
    average = total / len(scores)
    return(average)
def letter_grade(average):
    if average >= 90:
        letter = "A"
    elif average >= 80:
        letter = "B"
    elif average >= 70:
        letter = "C"
```

```
    elif average >= 60:
        letter = "D"
    else:
        letter = "F"
    return(letter)
def grade_calculator(scores):
    average = numeric_average(scores)
    letter = letter_grade(average)
    return average, letter

my_average, my_letter = grade_calculator(score_list)
print("Average:", my_average, "Letter grade:", my_letter)
```

Ron, this is beautiful work 🎯

You did exactly what this lesson is about:

numeric_average(scores) → one job (calculate + return average) ☑
letter_grade(average) → one job (decide and return letter) ☑
grade_calculator(scores) → just coordinates the helpers and returns both ☑
Printing is done outside the calculator, which keeps the functions reusable 👍

You fully applied one job per function and the code output looks correct:

Average: 88.75 Letter grade: B

Last quick understanding check, then we'll start wrapping this lesson:

If you later wanted to change how the message is printed (for example, different wording), which part would you edit now?

Just answer in your own words (e.g., "only the print at the bottom"), and we'll use that to lock in the concept.

Id only have to edit the print sttement to reformat the outout

Exactly ☑

Because the calculation and letter logic live in their own functions, you can change how the output looks by only touching this line:

Python

```
print("Average:", my_average, "Letter grade:", my_letter)
```

No risk of breaking the math or the grading rules. That's the power of one job per function: safer changes, less stress.

Let's zoom out for a second:

You understand that a function should have one clear responsibility
You can spot when a function is doing more than one thing (calculate + print, etc.)
You successfully split a bigger function into two helpers and a coordinator

You've hit all the goals for this lesson 🎓

Are you ready to wrap up this lesson and move on to the next one, or do you want one more quick example to practice spotting "too many jobs" in a function?

I'm ready to move on

Love it, Ron — strong work on this one 🎯

Quick recap of what you just mastered:

A function should have one clear responsibility (not “calculate + print + update + …” all in one).
You can spot functions doing too many jobs (like computing and printing together).
You can refactor by:
extracting small helpers (e.g., numeric_average, letter_grade)
using a coordinator function to call them
keeping printing / display separate from the logic
This is a big building block for writing clean, change-friendly code.

As you head into the next lesson, keep an eye out for any function you write and quietly ask yourself:
“Is this doing one job, or did I sneak in two or three?”

Whenever you want to practice more with splitting or cleaning up functions, I’m here to work through it with you 🚀